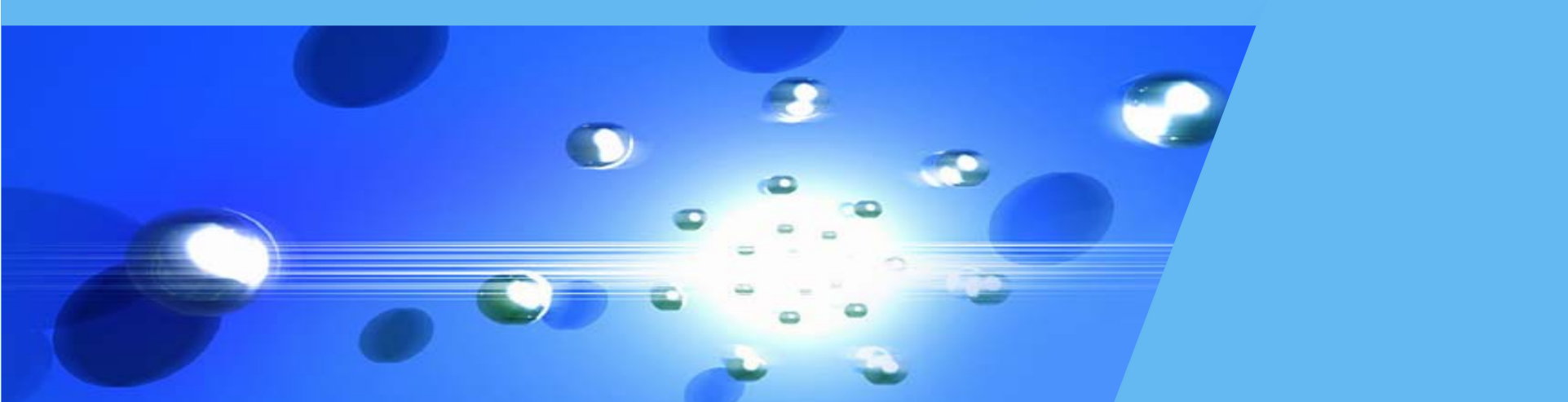


软件项目实训



第六章 Vue.js进阶

石 雷

合肥工业大学计算机与信息学院

课程提纲

1

组件基础

2

单文件组件

3

AJAX与Axios

4

过渡动画

5

路由Vue Router

6

状态管理

一. 组件基础

❖ 回到前面的两个问题

1. 一个网站很多页面，文字的字体、颜色等是否应该有个一致性？
2. 一个网站很多页面，但是有些部分是相同的，比如头部、底部，难道同样的代码写很多遍？

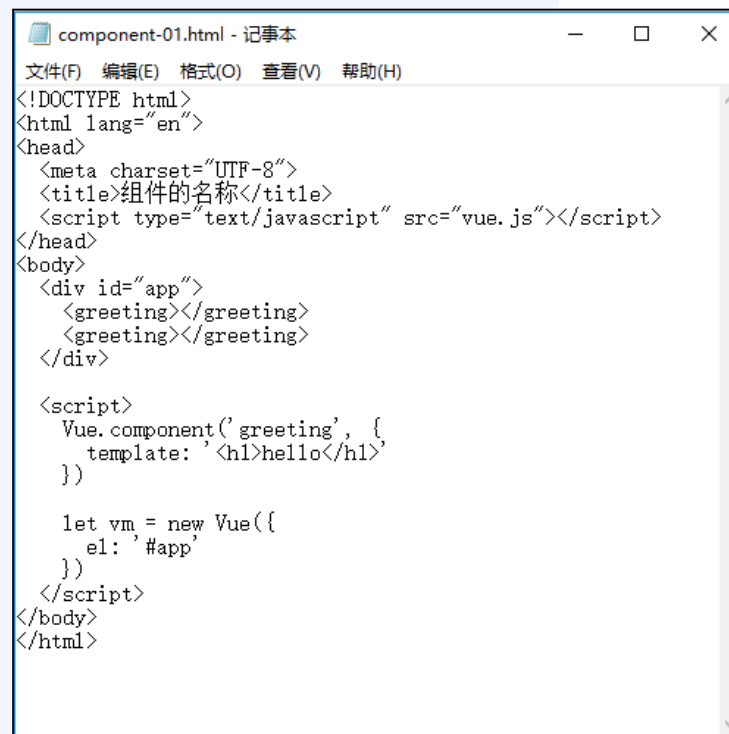


一. 组件基础

❖ 组件（**component**）

组件是在Vue.js中，由用户定义和实现的**可复用的Vue实例**，以实现开发过程中经常**重复使用的功能的封装**，从而达到便捷开发的目的。Vue中的组件可解决两类应用。

- 一些小的局部内容在网站的多个地方出现；
- 很多页面具有统一的整体页面布局形式。



```
component-01.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>组件的名称</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <greeting></greeting>
    <greeting></greeting>
  </div>

  <script>
    Vue.component('greeting', {
      template: '<h1>hello</h1>'
    })

    let vm = new Vue({
      el: '#app'
    })
  </script>
</body>
</html>
```

一. 组件基础

❖ 组件的组成部分

类似于一个HTML标记，组件有四个关键部分：

- 组件的名称 对应下面的a
- 组件的属性 对应下面的href
- 组件的内容 对应下面的 这是一个超链接
- 在组件中处理事件 对应下面的onClick

```
<a href="link.html" onclick="onClick()">这是一个超链接</a>
```

一. 组件基础

❖ 组件的名称

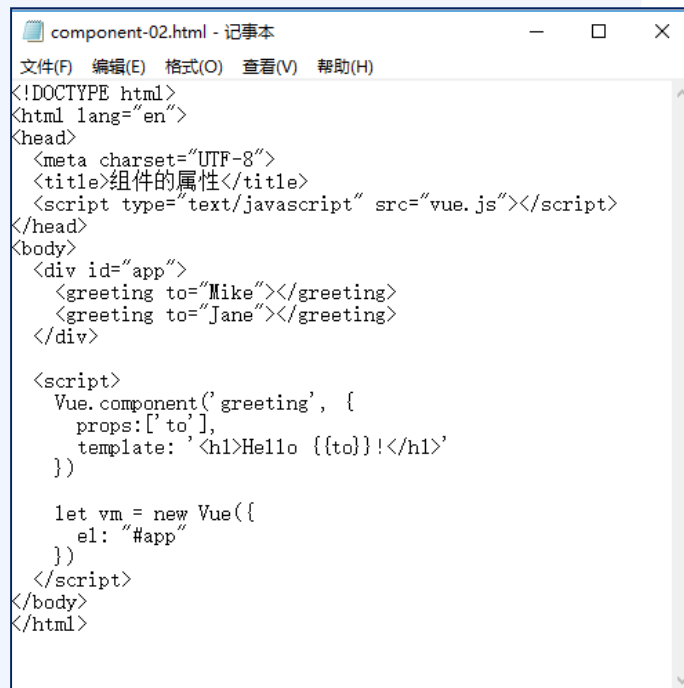
建议遵循kebab-case规范，即所有字母小写，名称中的单词之间用短横线连接。例如：monthly-calendar

补充知识：kebab-case、camelCase、Pascal Case

Vue.component('monthly-calendar', { /* ... */ })

❖ 组件的属性

可通过props定义组件的属性。



```
component-02.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>组件的属性</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <greeting to="Mike"></greeting>
    <greeting to="Jane"></greeting>
  </div>

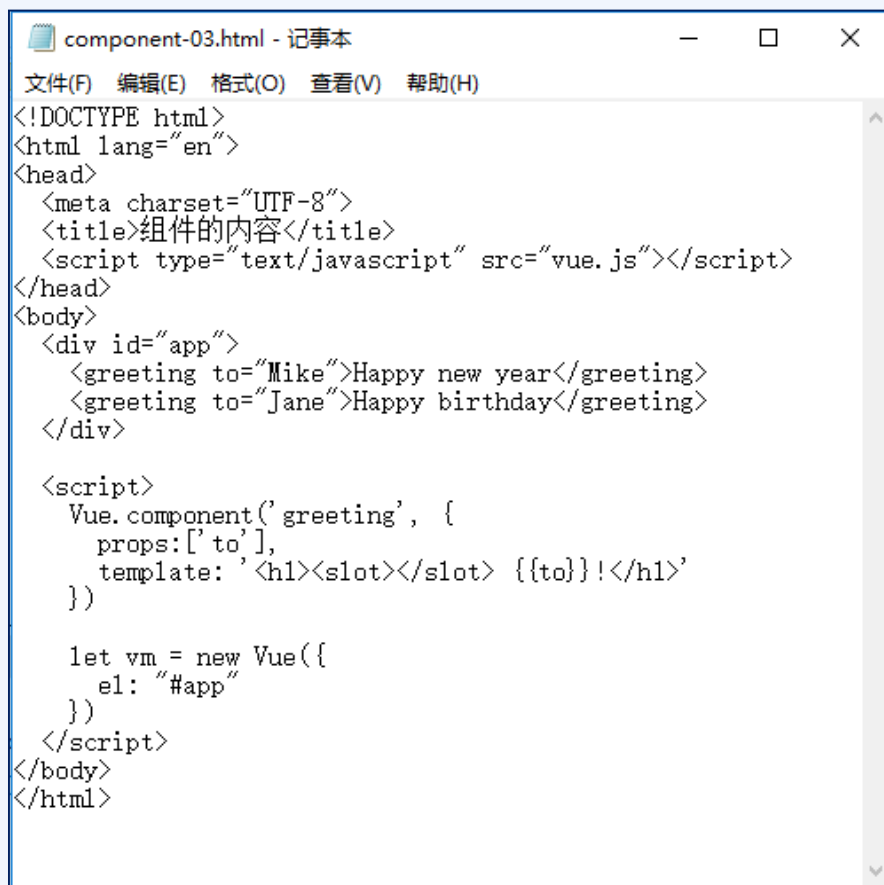
  <script>
    Vue.component('greeting', {
      props: ['to'],
      template: '<h1>Hello {{to}}!</h1>'
    })

    let vm = new Vue({
      el: "#app"
    })
  </script>
</body>
</html>
```

一. 组件基础

❖ 组件的内容

可通过slot（插槽）机制，灵活实现组件内容的设定。



```
component-03.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>组件的内容</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <greeting to="Mike">Happy new year</greeting>
    <greeting to="Jane">Happy birthday</greeting>
  </div>

  <script>
    Vue.component('greeting', {
      props: ['to'],
      template: '<h1><slot></slot> {{to}}!</h1>'
    })

    let vm = new Vue({
      el: "#app"
    })
  </script>
</body>
</html>
```

一. 组件基础

❖ 在组件中处理事件

```
component-04.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>在组件中处理事件1</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <greeting to="Mike">Love</greeting>
    <greeting to="Jane">Like</greeting>
  </div>

  <script>
    Vue.component('greeting', {
      data: function () {
        return {
          count: 0
        }
      },
      props: ['to'],
      template: '<button v-on:click="count++"><slot></slot> {{to}} x {{count}}</button>'
    })

    let vm = new Vue({
      el: "#app"
    })
  </script>
</body>
</html>
```

普通用法

```
component-05.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>在组件中处理事件2</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <greeting to="Mike">Love</greeting>
    <greeting to="Jane">Like</greeting>
  </div>

  <script>
    Vue.component('greeting', {
      data: function () {
        return {
          count: 0
        }
      },
      props: ['to'],
      methods: {
        onClick() {
          this.count++;
        }
      },
      template: '<button v-on:click="onClick"><slot></slot> {{to}} x {{count}}</button>'
    })

    let vm = new Vue({
      el: "#app"
    })
  </script>
</body>
</html>
```

事件逻辑写成单独的方法

```
component-06.html - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>在组件中处理事件3</title>
  <script type="text/javascript" src="vue.js"></script>
</head>
<body>
  <div id="app">
    <greeting to="Mike" v-on:click="onClick">Love</greeting>
    <greeting to="Jane">Like</greeting>
  </div>

  <script>
    Vue.component('greeting', {
      data: function () {
        return {
          count: 0
        }
      },
      props: ['to'],
      methods: {
        onClick() {
          this.count++;
          this.$emit('click', this.count);
        }
      },
      template: '<button v-on:click="onClick"><slot></slot> {{to}} x {{count}}</button>'
    })

    let vm = new Vue({
      el: "#app",
      methods: {
        onClick(count) {
          alert("已经点击了"+count+"次");
        }
      }
    })
  </script>
</body>
</html>
```

将组件中的事件暴露出来处理

课程提纲

1

组件基础

2

单文件组件

3

AJAX与Axios

4

过渡动画

5

路由Vue Router

6

状态管理

二. 单文件组件

❖ 基础准备——安装Vue CLI脚手架工具

第一步：安装node.js;

命令行提示符下确认: `node -v` `npm -v`

第二步：安装CLI;

`cnpm config set registry https://registry.npm.taobao.org`

`cnpm install @vue/cli -g`

第三步：创建工程;

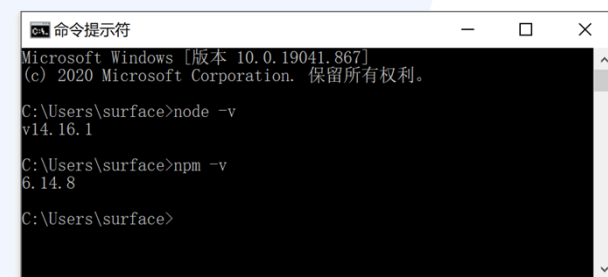
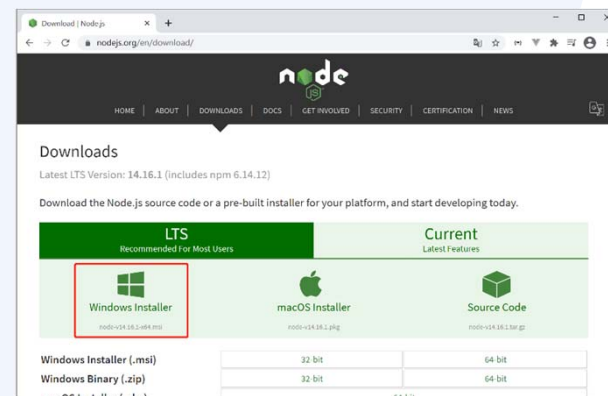
`md vue-projects`

`cd vue-projects`

`vue create my-first-app`

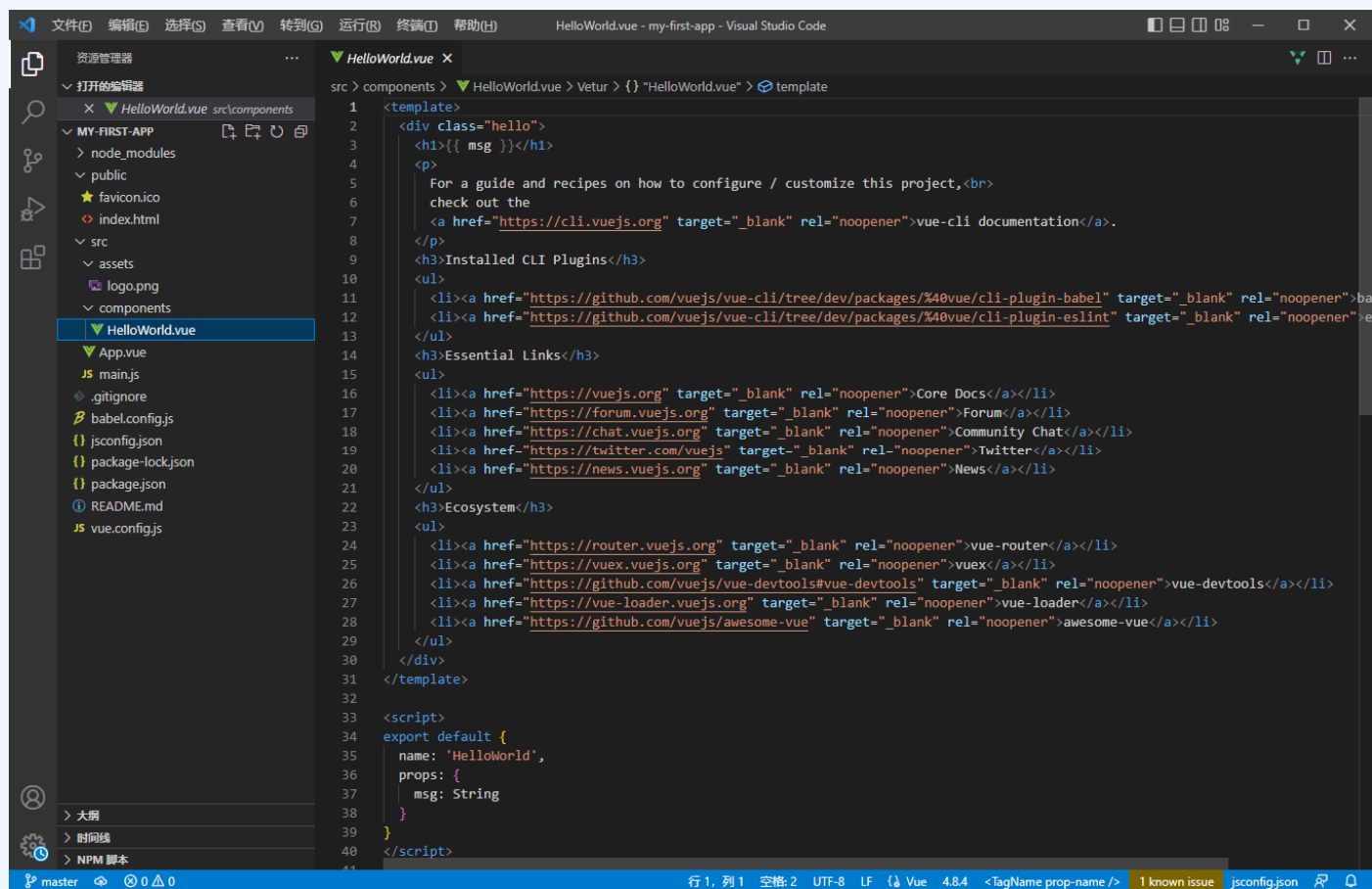
第四步：运行工程;

`npm run serve`



二. 单文件组件

❖ 基础准备——用VS code打开工程



二. 单文件组件

❖ 动手实现——投票页面



二. 单文件组件

❖ 单页应用和多页应用

	单页面	多页面
页面结构	一个页面 + 许多模块的组件	很多完整页面
体验效果	页面切换流畅，体验效果佳	页面切换慢，网速不好的时候，体验效果很不好
资源文件	组件公共的资源只需要加载一次	每个页面都要加载一次公共资源
路由模式	可以使用 hash ，也可以使用 history	普通链接跳转
适用场景	对体验效果和流畅度有较高要求的应用 不利于 SEO （搜索引擎收录，可借助服务器端渲染技术优化 SEO ）	适用于对 SEO 要求较高的应用
内容更新	相关组件的切换，即局部更新	整体 HTML 的切换
相关成本	前期开发成本较高，后期维护较为容易	前期开发成本低，后期维护就比较麻烦，可能一个功能需要改很多地方

二. 单文件组件

❖ 单页应用和多页应用



课程提纲

1

组件基础

2

单文件组件

3

AJAX与Axios

4

过渡动画

5

路由Vue Router

6

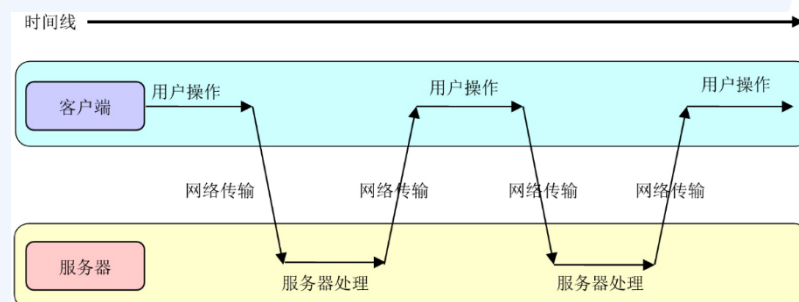
状态管理

三. AJAX和Axios

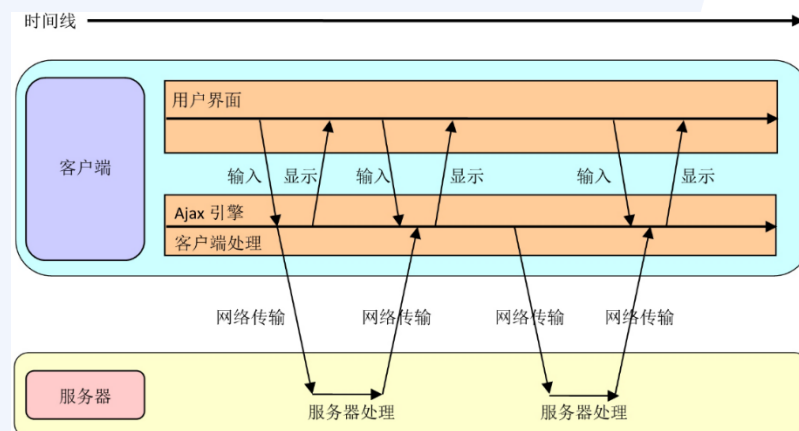
❖ AJAX

AJAX (asynchronous JavaScript and XML) 是为了创建交互性好的Web应用而诞生的技术, 采用JavaScript编写, 本质上是采用异步交互的方法实现Web交互, 最终优化用户体验。

传统同步交互



AJAX异步交互



三. AJAX和Axios

❖ AJAX的好处

- ✦ 减轻服务器的负担，加快浏览速度；
- ✦ 带来更好的用户体验；
- ✦ 基于标准化并被广泛支持的技术，不需要下载插件或小程序；
- ✦ 促进页面呈现与数据分离。

❖ AJAX的数据传输方式

- ✦ 一种是在服务器上直接生成HTML文档的片段，然后传递到浏览器上，进行局部更新。
- ✦ 另一种是服务器上生成的是数据，而不是文档，但是绝对大多数都是用JSON数据格式，而不是XML数据格式。

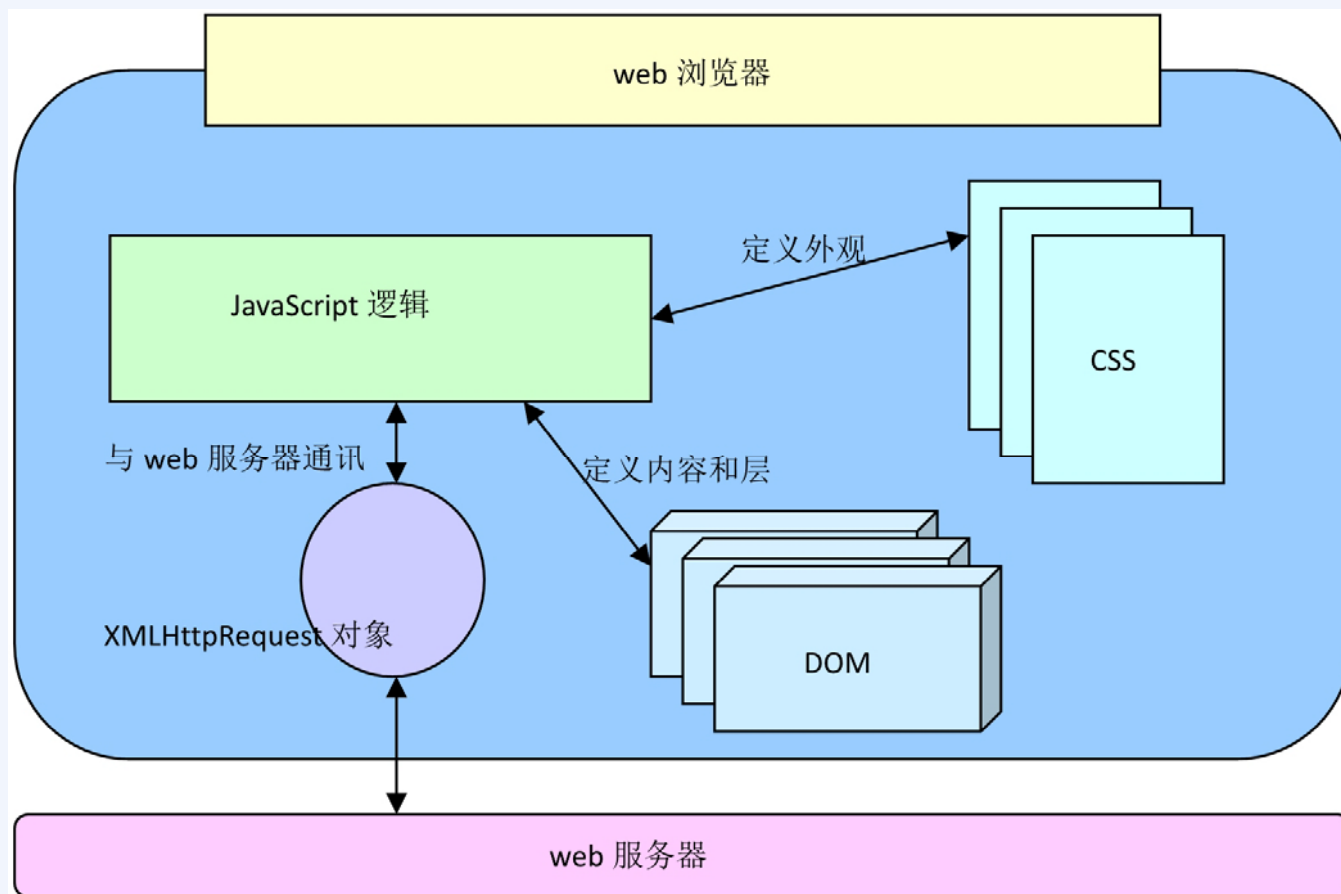
三. AJAX和Axios

❖ AJAX的组成

技术	角色
JavaScript	JavaScript 是通用的脚本语言，用来嵌入在某种应用之中。 Ajax 应用程序是使用 JavaScript 编写的。
CSS	CSS 为 Web 页面元素提供了可视化样式的定义方法。 Ajax 应用中，用户界面的样式可以通过 CSS 独立修改。
DOM	通过 JavaScript 修改 DOM ， Ajax 应用程序可以在运行时改变用户界面，或者局部更新页面中的某个节点。
XMLHttpRequest对象	XMLHttpRequest 对象允许开发人员从 Web 服务器上获取数据。数据的格式通常是 JSON 、 XML ，或者文本。

三. AJAX和Axios

❖ AJAX的组成



三. AJAX和Axios

❖ 用AJAX原生方法获取异步数据

```
let xmlHttp = new XMLHttpRequest();
xmlHttp.open(
  "GET",
  "http://demo-api.geekfun.website/vue/ajax-test.aspx",
  true
);
xmlHttp.onreadystatechange = () => {
  if(xmlHttp.readyState == 4 && xmlHttp.status == 200)
    this.msg = xmlHttp.responseText;
};
xmlHttp.send(null);
```

<http://demo-api.geekfun.website/vue/ajax-test.aspx>

三. AJAX和Axios

❖ Axios

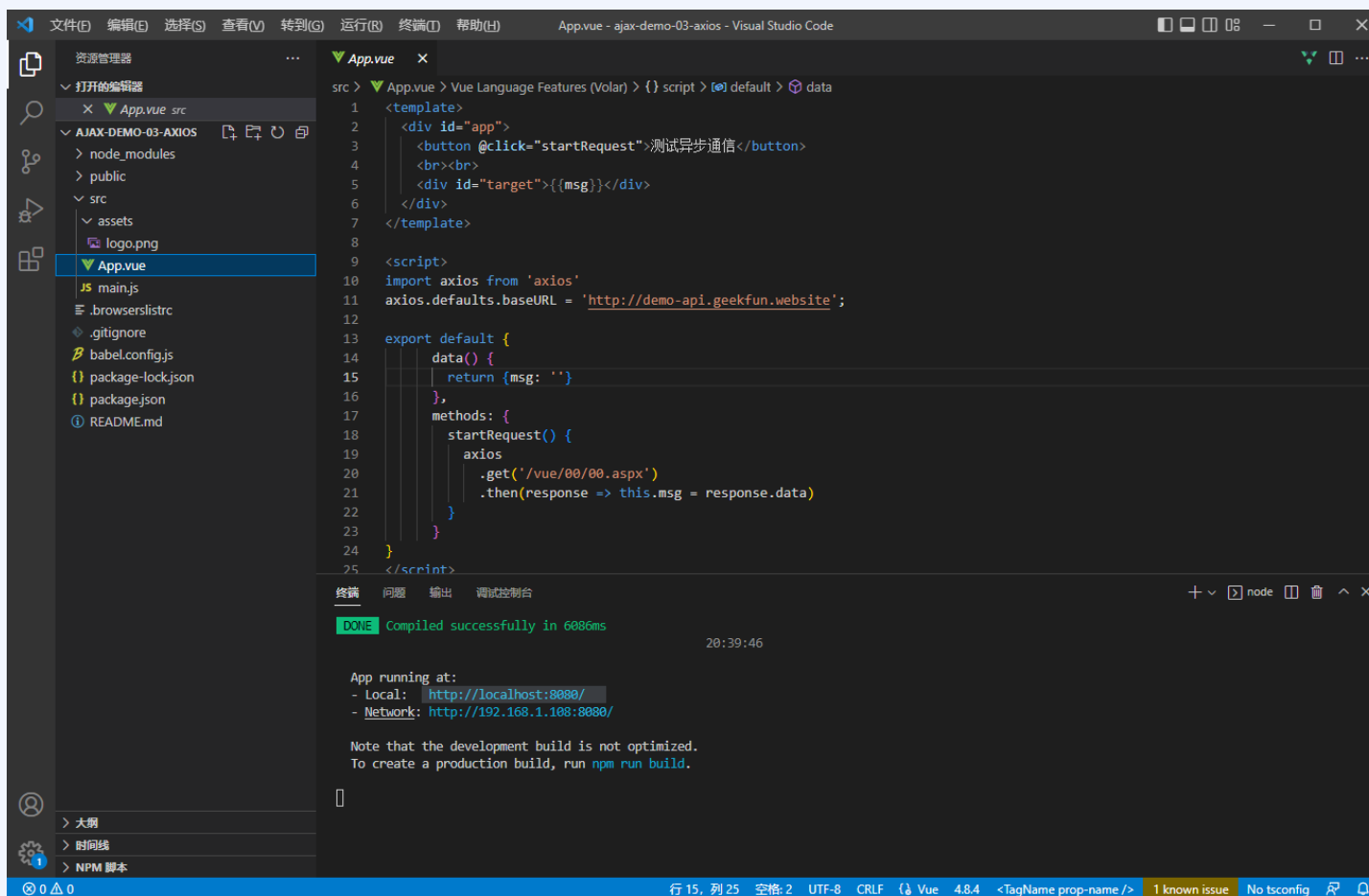
Axios是一个专门用来处理AJAX相关工作的库。将Axios和Vue.js搭配后，可以方便地在Web项目中使用AJAX技术。

```
npm install axios --save
```

在页面中引入**axios.js**文件

三. AJAX和Axios

❖ Axios



```
src > App.vue x
src > App.vue > Vue Language Features (Volar) > {} script > [0] default > data
1 <template>
2   <div id="app">
3     <button @click="startRequest">测试异步通信</button>
4     <br><br>
5     <div id="target">{{msg}}</div>
6   </div>
7 </template>
8
9 <script>
10 import axios from 'axios'
11 axios.defaults.baseURL = 'http://demo-api.geekfun.website';
12
13 export default {
14   data() {
15     return {msg: ''}
16   },
17   methods: {
18     startRequest() {
19       axios
20         .get('/vue/00/00.aspx')
21         .then(response => this.msg = response.data)
22     }
23   }
24 }
25 </script>
```

终端 问题 输出 调试控制台

DONE Compiled successfully in 6086ms 20:39:46

App running at:

- Local: <http://localhost:8080/>
- Network: <http://192.168.1.108:8080/>

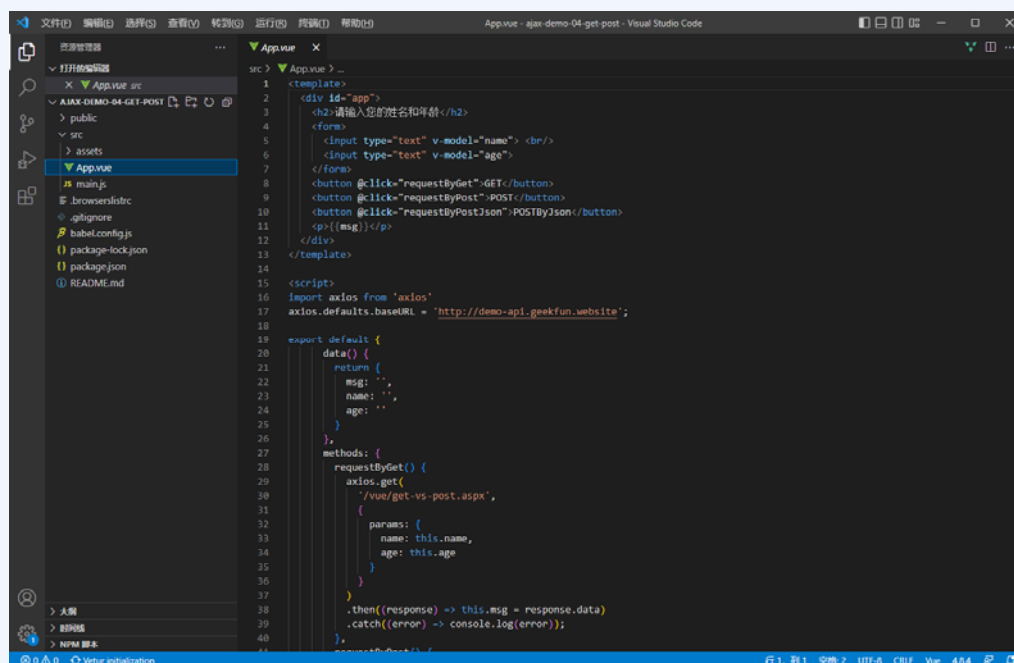
Note that the development build is not optimized.
To create a production build, run `npm run build`.

行 15, 列 25 空格: 2 UTF-8 CRLF (Vue 4.8.4 <TagName prop-name /> 1 known issue No tsconfig)

三. AJAX和Axios

❖ GET和POST方法

- ⊕ GET方式发起请求时，参数会以查询字符串的形式，作为URL的一部分传递给服务器，服务器收到请求后，会解析查询字符串，得到请求的参数。
- ⊕ POST方式发起请求时，参数会以Form Data方式出现在请求的报文正文中。



```
1 <template>
2 <div id="app">
3   <h2>请输入您的姓名和年龄</h2>
4   <form>
5     <input type="text" v-model="name"> <br/>
6     <input type="text" v-model="age">
7   </form>
8   <button @click="requestByGet">GET</button>
9   <button @click="requestByPost">POST</button>
10  <button @click="requestByPostJson">POSTByJson</button>
11  <p>{{msg}}</p>
12 </div>
13 </template>
14
15 <script>
16 import axios from 'axios'
17 axios.defaults.baseURL = 'http://demo-api.geekfun.website';
18
19 export default {
20   data() {
21     return {
22       msg: '',
23       name: '',
24       age: ''
25     }
26   },
27   methods: {
28     requestByGet() {
29       axios.get(
30         '/vue/get-vs-post.aspx',
31         {
32           params: {
33             name: this.name,
34             age: this.age
35           }
36         }
37       )
38       .then(response => this.msg = response.data)
39       .catch(error => console.log(error));
40     },
41     requestByPost() {
42       // ...
43     }
44   }
45 }
```

三. AJAX和Axios

❖ 嵌套请求和并发请求

嵌套请求:

```
//外层
Axios.get(URL)
.then(
  //内层
  (response) =>
    Axios.get(URL, 参数)
    .then((response) => {
      //请求成功后要执行的内容
    })
  )
)
```

并发请求:

```
Axios.all([
  Axios.get(get1),
  Axios.get(get2)
]).then(
  Axios.spread((Res1, Res2) => {
    console.log(Res1, Res2)
  })
)
```


三. AJAX和Axios

❖ Axios进阶用法——创建实例

```
const axios = axios.create({  
  baseURL: 'http://localhost:8080',  
  timeout: 1000, //设置超时时长，请求未返回超过一秒就算超时  
  //其他配置项.....  
});
```

配置项	举例	说明
url	'/user'	用于请求服务器的URL
method	'get'	创建请求时使用的方法，还可以是'post', 'put', 'patch', 'delete', 默认是get
baseURL	'http://localhost:8080'	将自动加在url前面，除非url是一个绝对URL
headers	{'content-type': 'application/x-www-form-urlencoded'}	是即将被发送的自定义请求头
params	{ id: 1 }	请求参数拼接在url后面
data	{ id: 1 }	请求参数放在请求体里
timeout	1000	指定请求超时的毫秒数(0 表示无超时时间)，如果请求花费了超过timeout的时间，请求将被中断，为了不阻塞后面要执行的内容
responseType	'json'	表示希望服务器响应的数据类型，还可以是 'arraybuffer', 'blob', 'document', 'text', 'stream'，默认是json

三. AJAX和Axios

❖ Axios进阶用法——错误处理

❖ 错误处理

- 请求异常
- 响应异常

❖ 拦截器

- 请求拦截器
- 响应拦截器

```
this.axios.interceptors.request.use(  
  config => config,  
  error => {  
    this.showError('请求异常')  
    return Promise.reject(error);  
  }  
);  
  
this.axios.interceptors.response.use(  
  response => response,  
  error => {  
    this.showError('响应异常: ' + error.message);  
    return Promise.reject(error);  
  }  
);
```

课程提纲

1

组件基础

2

单文件组件

3

AJAX与Axios

4

过渡动画

5

路由Vue Router

6

状态管理

四. 过渡动画

过渡动画能够使网页更加生动。**Vue.js**提供了多种不同方式的过渡效果：**CSS过渡**、**单元素过渡**、**列表过渡**。

教学相关 > 教学课件 > 上课常用 > 网站设计开发 > 课件第二版 > 第六章例子 > 第12章			
名称	修改日期	类型	大小
01.html	2021/6/9 18:27	Chrome HTML D...	1 KB
02.html	2021/6/9 18:27	Chrome HTML D...	1 KB
03.html	2021/6/9 18:27	Chrome HTML D...	2 KB
menu-01.html	2021/7/1 18:44	Chrome HTML D...	2 KB
menu-02.html	2021/7/1 18:44	Chrome HTML D...	2 KB
menu-03.html	2021/7/1 18:44	Chrome HTML D...	3 KB
menu-04.html	2021/7/1 18:44	Chrome HTML D...	3 KB
menu-05.html	2021/7/1 18:44	Chrome HTML D...	3 KB
todo-01.html	2021/6/9 18:27	Chrome HTML D...	2 KB
todo-02.html	2021/6/9 18:27	Chrome HTML D...	2 KB
todo-03.html	2021/6/9 18:27	Chrome HTML D...	2 KB
todo-04.html	2021/6/9 18:27	Chrome HTML D...	3 KB
todo-05.html	2021/6/9 18:27	Chrome HTML D...	3 KB
vue.js	2021/6/9 18:27	JavaScript 文件	346 KB

课程提纲

1

组件基础

2

单文件组件

3

AJAX与Axios

4

过渡动画

5

路由Vue Router

6

状态管理

五. 路由Vue Router

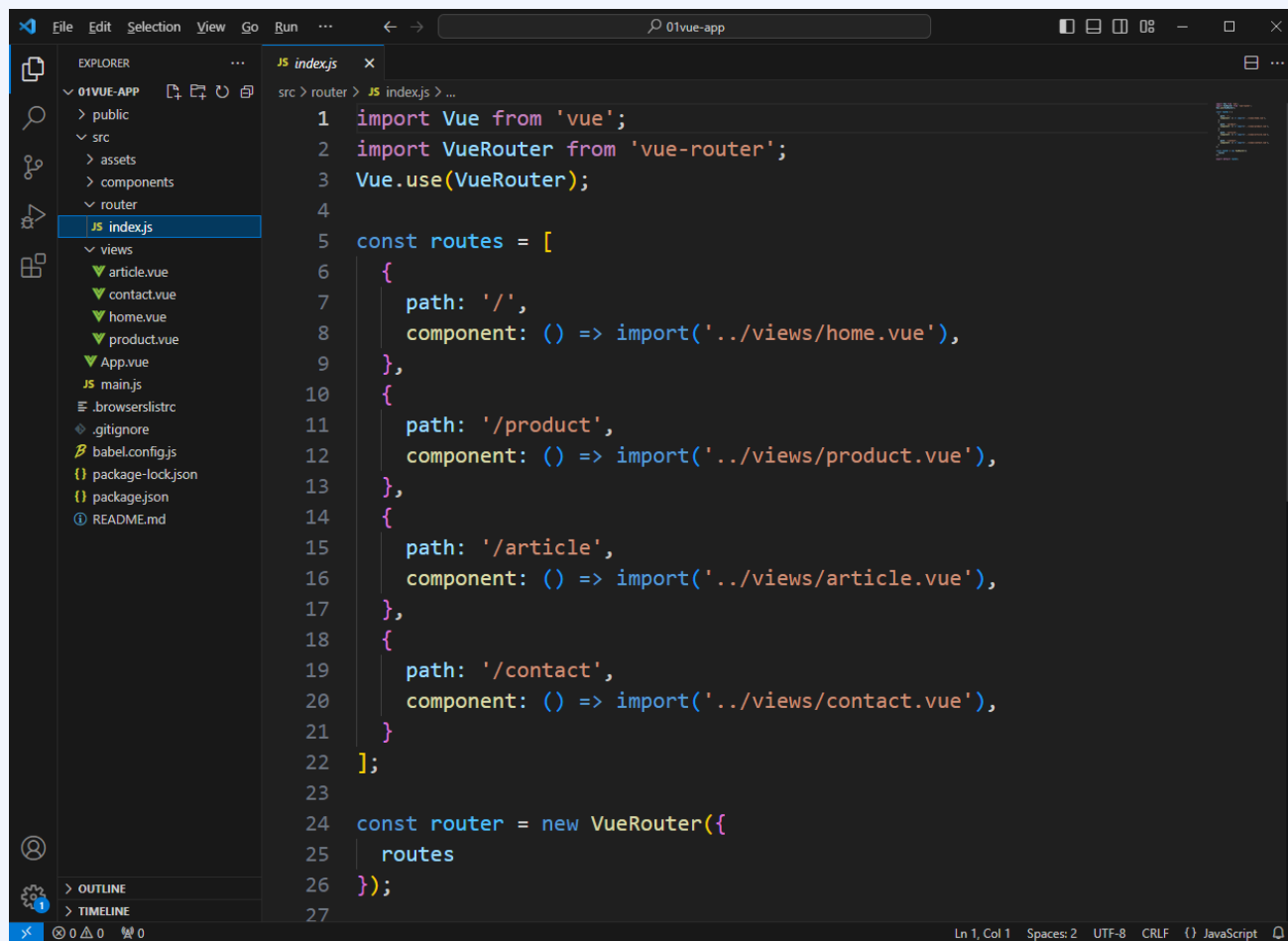
❖ 路由Vue Router

路由是Vue.js中，根据URL来匹配对应的组件，并且无刷新切换模板内容的一种方式。简单地说，路由实现一个Web系统中不同页面跳转的管理。

- 第一步：安装Vue Router； `npm install vue-router`
- 第二步：通过Vue.use()使用路由，并在src文件夹下创建router文件夹和相关文件；
- 第三步：在router/index.js路由文件中配置路由；
- 第四步：将创建好的路由挂载到Vue实例上；
- 最后：在相关页面位置使用路由。

五. 路由Vue Router

❖ 基本用法

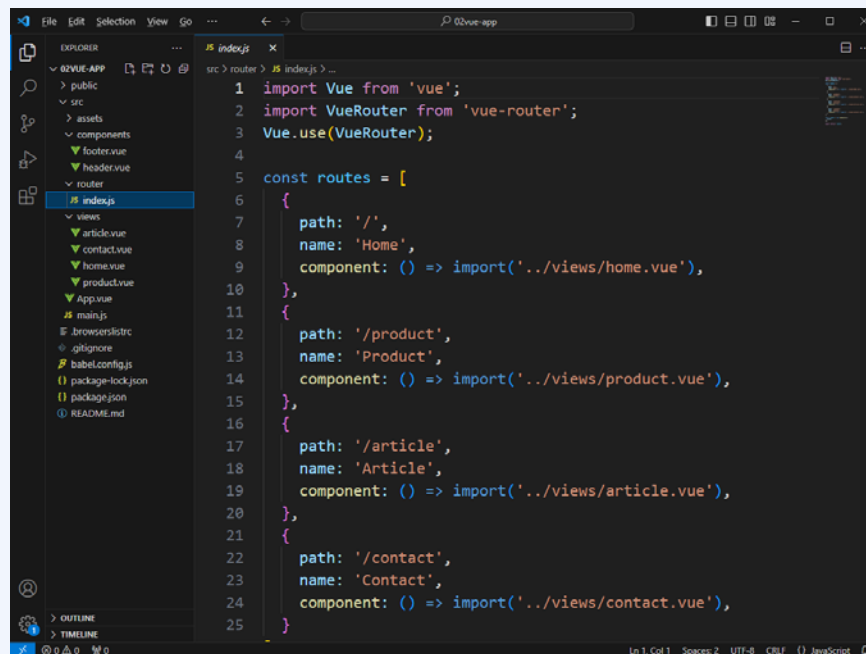


```
1 import Vue from 'vue';
2 import VueRouter from 'vue-router';
3 Vue.use(VueRouter);
4
5 const routes = [
6   {
7     path: '/',
8     component: () => import('../views/home.vue'),
9   },
10  {
11    path: '/product',
12    component: () => import('../views/product.vue'),
13  },
14  {
15    path: '/article',
16    component: () => import('../views/article.vue'),
17  },
18  {
19    path: '/contact',
20    component: () => import('../views/contact.vue'),
21  }
22 ];
23
24 const router = new VueRouter({
25   routes
26 });
27
```

五. 路由Vue Router

❖ 命名路由

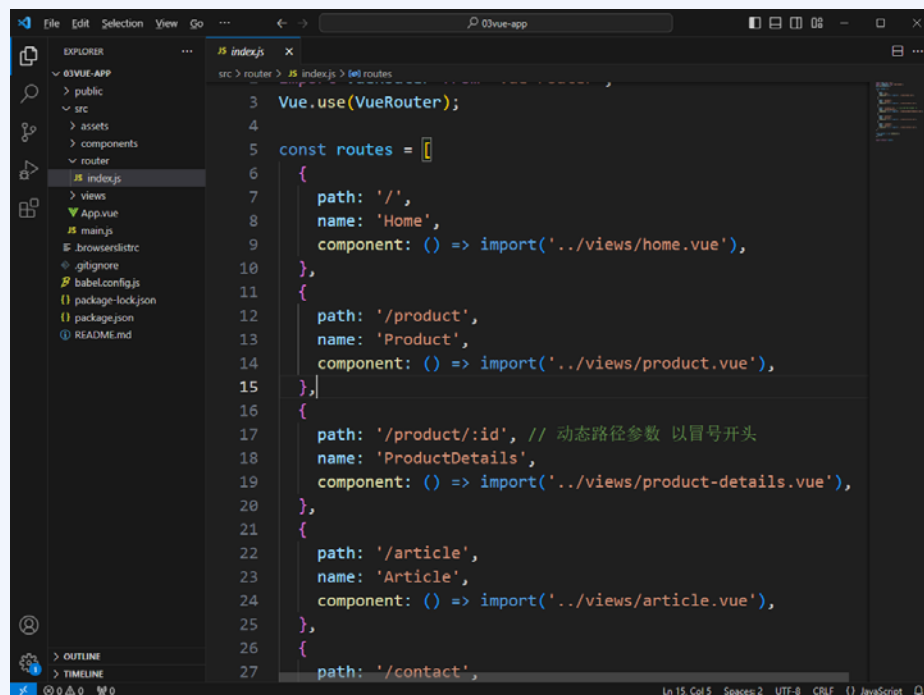
```
{  
  path: '/',  
  name: 'Home', //使用name属性给路由命名  
  component: () => import('../views/home.vue')  
}
```



五. 路由Vue Router

❖ 路由参数

```
{  
  path: '/product/:id',    // 动态路径参数以冒号开头  
  name: 'ProductDetails',  
  component: () => import('../views/product-details.vue')  
}
```



五. 路由Vue Router

❖ 多路由参数与侦听路由

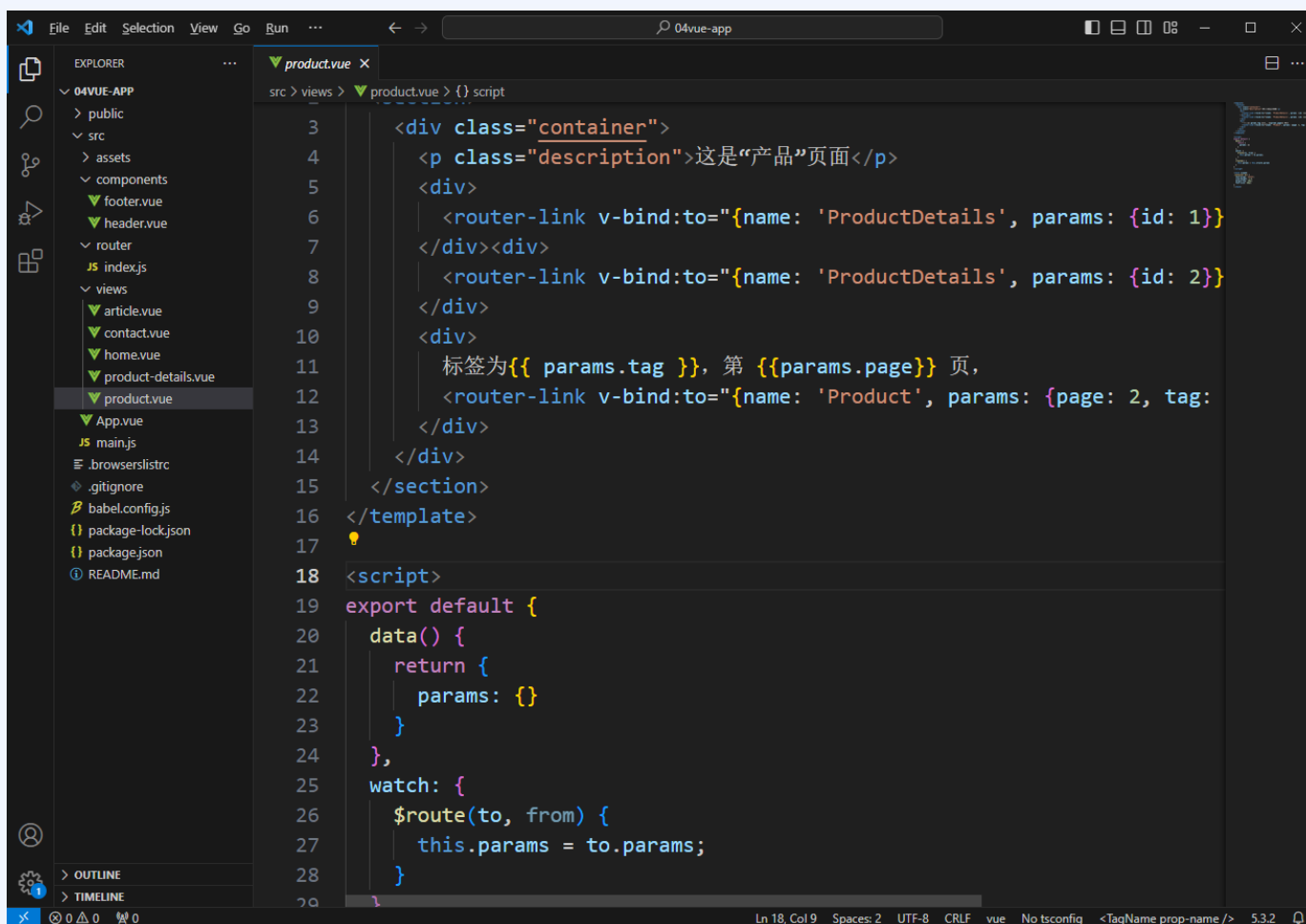
```
{  
  path: '/product/:page?/:tag?',  
  name: 'Product',  
  component: () => import('../views/product.vue')  
}
```

模式	匹配路径	\$route.params
/product/:id	/product/1	{id: 1}
/product/:page/:tag	/product/1/0	{page: 1, tag: 0}

```
watch: {  
  $route(to, from) {  
    this.params = to.params;  
  }  
}
```

五. 路由Vue Router

❖ 多路由参数与侦听路由



```
3  <div class="container">
4    <p class="description">这是“产品”页面</p>
5    <div>
6      <router-link v-bind:to="{name: 'ProductDetails', params: {id: 1}}>
7    </div><div>
8      <router-link v-bind:to="{name: 'ProductDetails', params: {id: 2}}>
9    </div>
10   <div>
11     标签为{{ params.tag }}, 第 {{params.page}} 页,
12     <router-link v-bind:to="{name: 'Product', params: {page: 2, tag:
13   </div>
14 </div>
15 </section>
16 </template>
17
18 <script>
19 export default {
20   data() {
21     return {
22       params: {}
23     }
24   },
25   watch: {
26     $route(to, from) {
27       this.params = to.params;
28     }
29   }
30 }
```

五. 路由Vue Router

❖ 查询参数

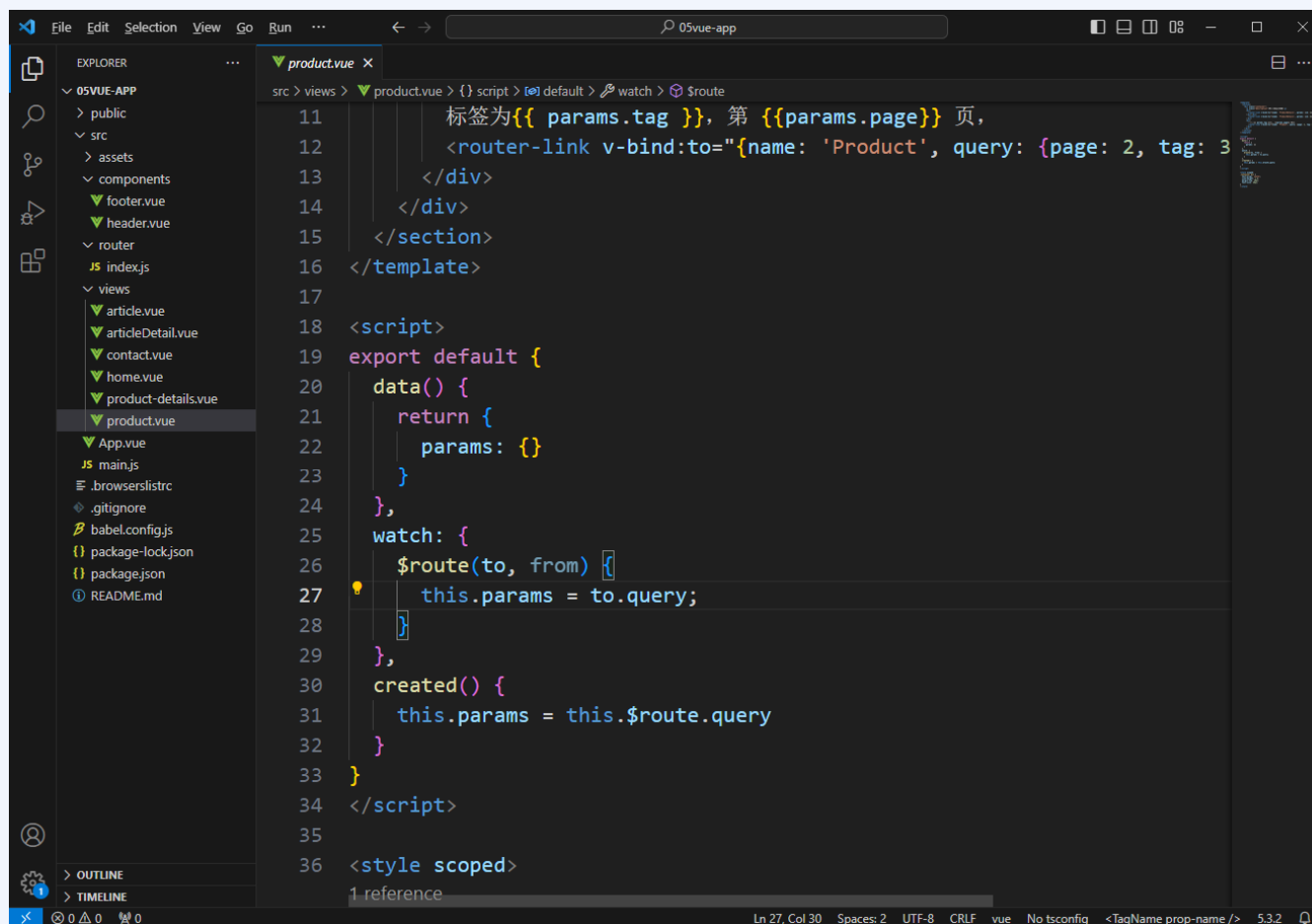
```
<router-link v-bind:to="{name: 'Product', query: {page: 2, tag: 3}}">下一页</router-link>
```

```
watch: {  
  $route(to, from) {  
    this.params = to.query;  
  },  
  created() {  
    this.params = this.$route.query  
  }  
}
```

```
http://localhost:8080/#/product?page=2&tag=3
```

五. 路由Vue Router

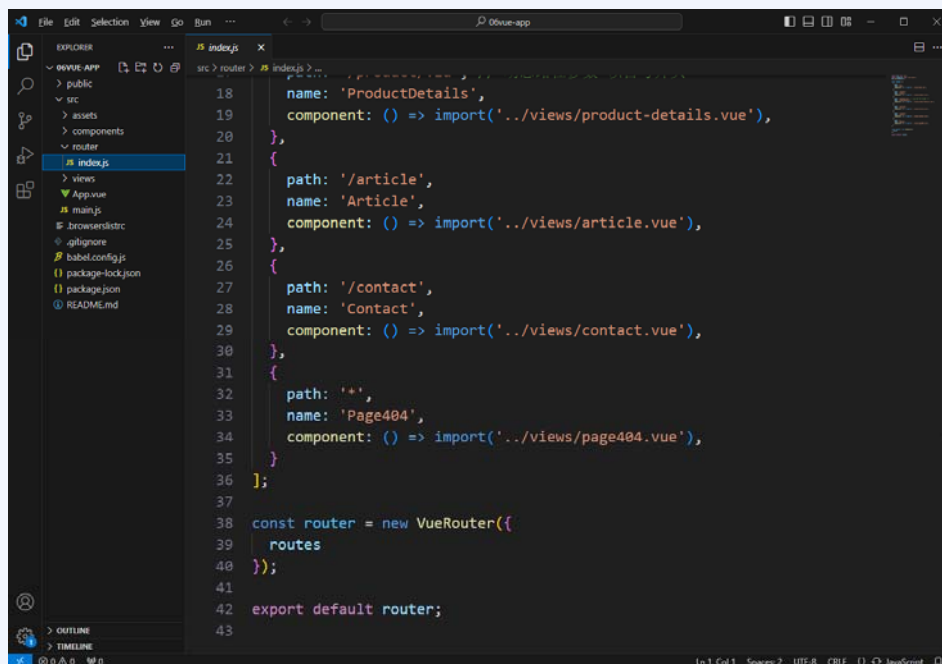
❖ 查询参数



五. 路由Vue Router

❖ 捕获所有路由（404）

```
{  
  path: '*',  
  name: 'Page404',  
  component: () => import('../views/page404.vue')  
}
```



五. 路由Vue Router

❖ 编程式导航

```
// 字符串
router.push('home')

// 对象
router.push({ path: 'home' })

// 命名的路由
router.push({ name: 'product', params: { id: '123' } })

// 带查询参数，变成 /register?plan=private
router.push({ path: 'register', query: { plan: 'private' } })

const id = '123'
router.push({ name: 'product', params: { id } }) // -> /product/123
router.push({ path: `/product/${id}` }) // -> /product/123
// 这里的 params 不生效
router.push({ path: '/product', params: { id } }) // -> /product
```

五. 路由Vue Router

❖ 重定向和别名

```
const routes = [  
  { path: '/a', redirect: '/b' }, //字符串路径  
  { path: '/a', redirect: { name: 'foo' } }, //路径对象  
  { path: '/a', redirect: to => {  
    // 方法接收 目标路由 作为参数  
    // return 重定向的 字符串路径/路径对象  
  } } ]
```

```
{  
  path: '/product/:id',  
  name: 'ProductDetails',  
  component: () => import('../views/product-detail.vue'),  
  alias: '/product/details/:id'  
}
```


五. 路由Vue Router

❖ 导航守卫

全局前置守卫：
`router.beforeEach(function(to, from, next) {`
 //要执行的内容
`})`

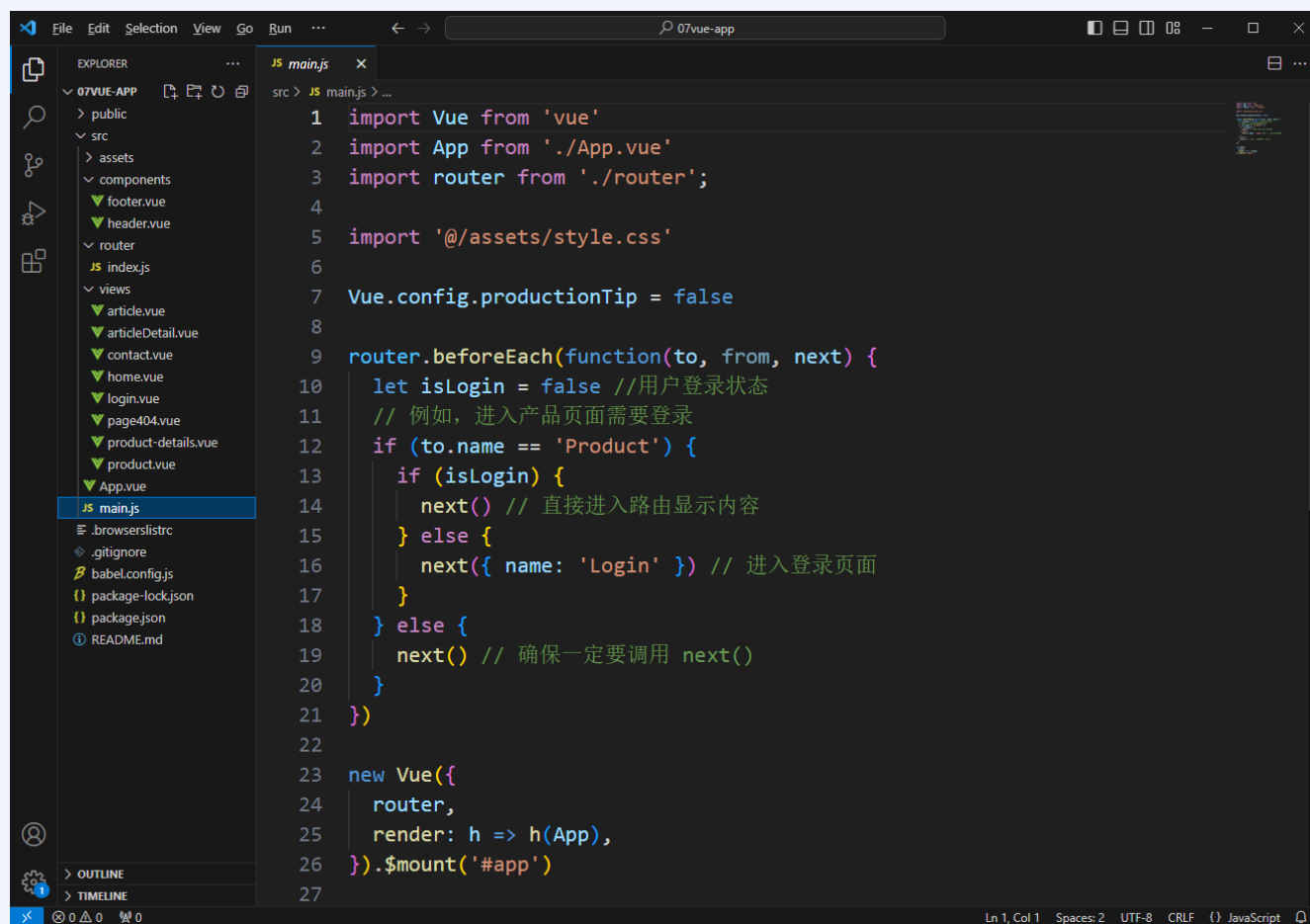
每个守卫方法接收三个参数：

- to: 即将要进入的路由
- from: 当前导航正要离开的路由
- next: 一定要调用该方法来resolve这个钩子。执行效果依赖next方法的调用参数。
 - next()
 - next(false)
 - next('/')或者next({ path: '/' })
 - next(error)

路由中配置前置守卫：
`{`
 path: '/product',
 name: 'Product',
 component: Product,
 beforeEnter: (to, from, next) => {
 // ...
 }
`}`

五. 路由Vue Router

❖ 导航守卫



```
1 import Vue from 'vue'
2 import App from './App.vue'
3 import router from './router';
4
5 import '@/assets/style.css'
6
7 Vue.config.productionTip = false
8
9 router.beforeEach(function(to, from, next) {
10   let isLogin = false //用户登录状态
11   // 例如, 进入产品页面需要登录
12   if (to.name === 'Product') {
13     if (isLogin) {
14       next() // 直接进入路由显示内容
15     } else {
16       next({ name: 'Login' }) // 进入登录页面
17     }
18   } else {
19     next() // 确保一定要调用 next()
20   }
21 })
22
23 new Vue({
24   router,
25   render: h => h(App),
26 }).$mount('#app')
```

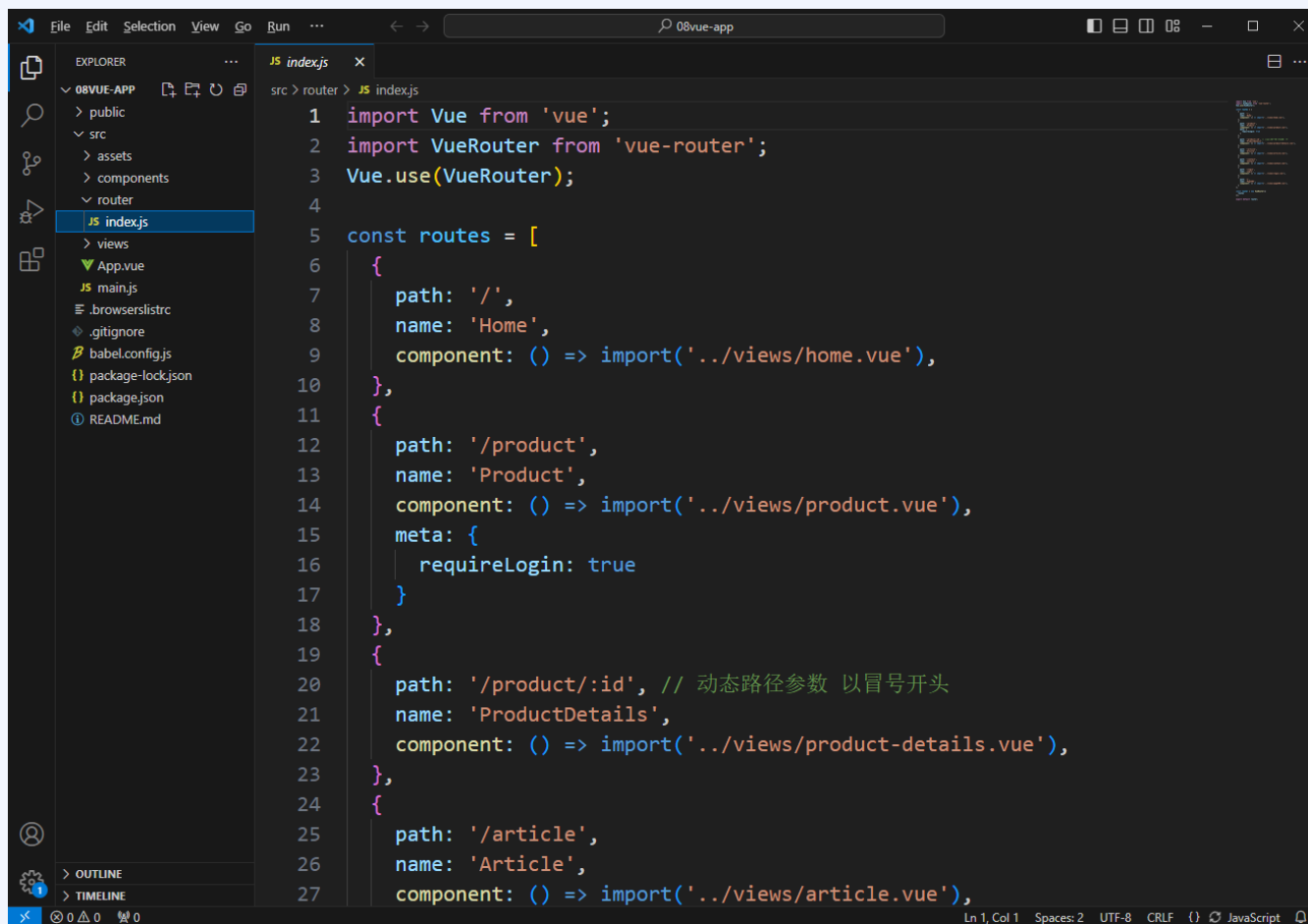
五. 路由Vue Router

❖ 路由元信息

```
const routes = [  
  {  
    path: '/product',  
    name: 'Product',  
    component: () => import('../views/product.vue'),  
    meta: {  
      requireLogin: true //为true表示需要登录才能访问  
    }  
  },  
  ...省略...  
];
```

五. 路由Vue Router

❖ 路由元信息



```
1 import Vue from 'vue';
2 import VueRouter from 'vue-router';
3 Vue.use(VueRouter);
4
5 const routes = [
6   {
7     path: '/',
8     name: 'Home',
9     component: () => import('../views/home.vue'),
10  },
11  {
12    path: '/product',
13    name: 'Product',
14    component: () => import('../views/product.vue'),
15    meta: {
16      requireLogin: true
17    }
18  },
19  {
20    path: '/product/:id', // 动态路径参数 以冒号开头
21    name: 'ProductDetails',
22    component: () => import('../views/product-details.vue'),
23  },
24  {
25    path: '/article',
26    name: 'Article',
27    component: () => import('../views/article.vue'),
```

五. 路由Vue Router

❖ history模式

```
const router = new VueRouter({  
  mode: 'history',  
  routes: [...]  
})
```

hash模式	history模式
http://localhost:8080/#/product	http://localhost:8080/product

课程提纲

1

组件基础

2

单文件组件

3

AJAX与Axios

4

过渡动画

5

路由Vue Router

6

状态管理

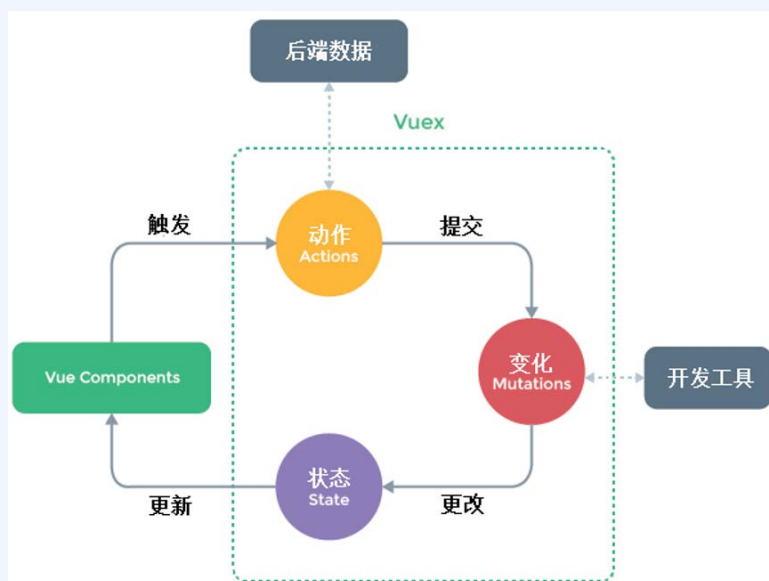
六. 状态管理

❖ 状态

状态就是数据，例如父子组件之间通过props和 \$emit 实现数据共享。

❖ 状态管理

当某些数据在不同组件不同地方大量共享，则需要有一种专门用来集中管理整个应用动态的机制，这种机制被抽离出来构成了一个组件Vuex。



六. 状态管理

❖ store模式

- ✦ 编写整体页面结构
- ✦ 创建store对象
- ✦ 使用store对象

购物车中商品数量: 2

华为Mate40 Pro

iPhone 12 pro

小米11 Ultra

vivo S9

iPhone 12 pro

华为Mate40 Pro

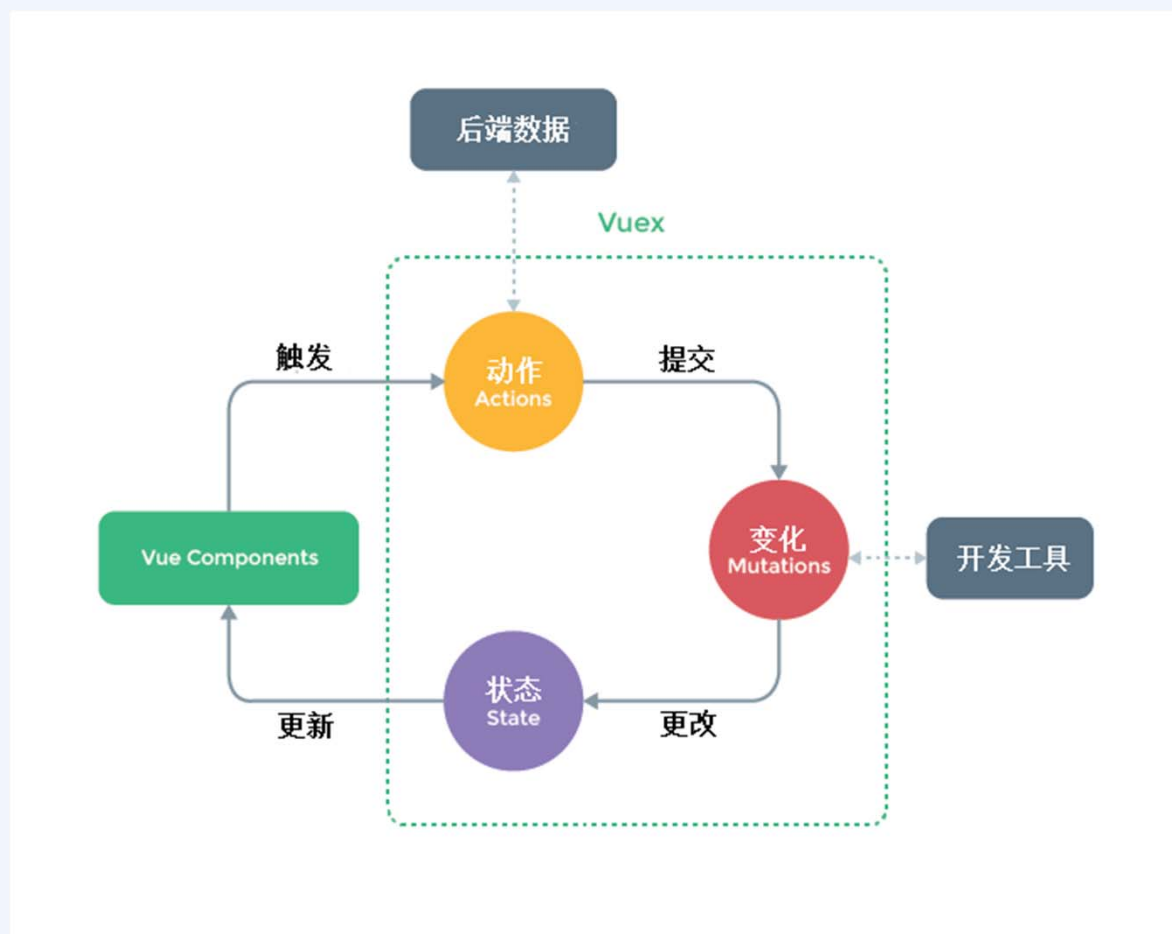
```
shopping.html
48 <product name="iPhone 12 pro"></product>
49 <product name="小米11 Ultra"></product>
50 <product name="vivo S9"></product>
51 </div>
52 <cart></cart>
53 </div>
54 <script>
55   let store = {
56     state:{
57       products:[]
58     },
59     getProducts(){
60       return this.state.products;
61     },
62     addToCart(name){
63       if(!this.state.products.includes(name))
64         this.state.products.push(name);
65     },
66     checkout(){
67       //.....这里省略下订单的逻辑.....
68       this.state.products=[];
69     }
70   };
71
72   let product = Vue.component("product", {
```


六. 状态管理

❖ Vuex

Vuex 是独立于Vue.js的一个专门为Vue.js应用程序开发的**状态管理插件**。通常情况下，每个组件都拥有自己的状态。有时，需要将某个组件的状态变化传递到其他组件，使它们也进行相应的修改。这时就可以使用Vuex保存需要管理的状态值，状态值一旦被修改，所有引用状态值的组件就会自动进行更新。Vuex采用**集中式存储**管理应用的所有组件的状态，并以相应的规则保证状态以一种可预测的方式发生变化。

六. 状态管理



六. 状态管理



```
56 </div>
57 <script>
58   const store = new Vuex.Store({
59     state:{
60       products:[]
61     },
62     getters:{
63       products(state){
64         return state.products;
65       }
66     },
67     mutations: {
68       addToCart(state, name){
69         if(!state.products.includes(name))
70           state.products.push(name);
71       },
72       checkOut(state){
73         //.....这里省略下订单的逻辑.....
74         state.products=[];
75       }
76     }
77   });
78
79   const product = Vue.component("product", {
80     props: ['name'],
```

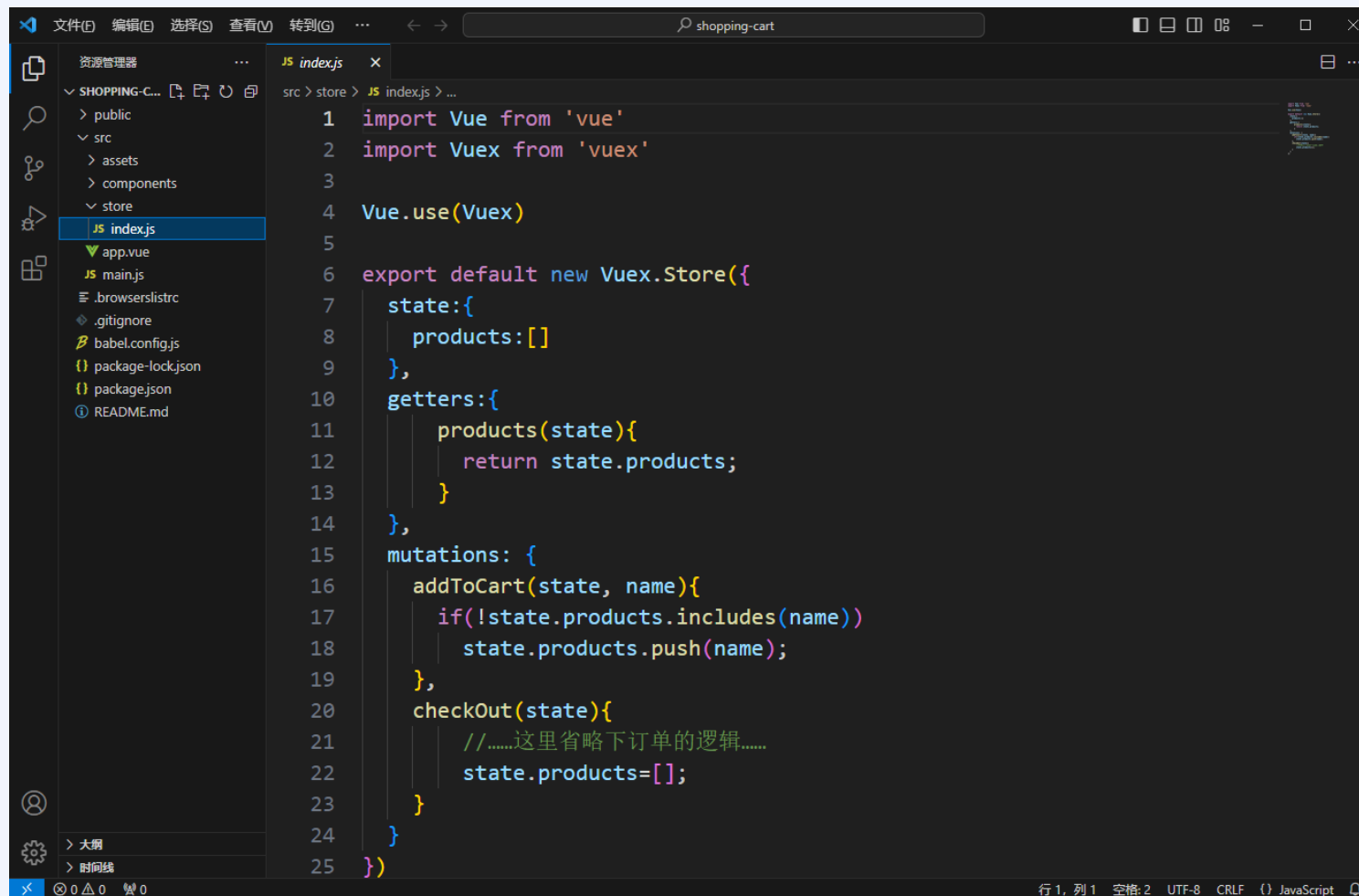
六. 状态管理

❖ 使用Vuex的简单步骤

1. 通过**Vue.Store()**方法创建**store**实例，在**store**实例的**state**属性中定义需要共享的状态，在**getters**中定义需要读取的状态或者状态经过计算以后的结果，在**mutations**对象中定义对状态执行的修改操作。
2. 在根实例中注入上一步创建的 **store**对象。
3. 在所有需要使用**store**对象的组件中，通过**this.\$store**获取**store**对象，然后通过**getters**读取状态，并通过**commit()**方法提交对状态执行的修改操作。

六. 状态管理

❖ 在单文件组件中使用Vuex



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with a 'store' directory containing 'index.js'. The code editor displays the content of 'index.js', which sets up a Vuex store with a state, getters, and mutations.

```
1 import Vue from 'vue'
2 import Vuex from 'vuex'
3
4 Vue.use(Vuex)
5
6 export default new Vuex.Store({
7   state: {
8     products: []
9   },
10  getters: {
11    products(state) {
12      return state.products;
13    }
14  },
15  mutations: {
16    addToCart(state, name) {
17      if (!state.products.includes(name)) {
18        state.products.push(name);
19      }
20    },
21    checkout(state) {
22      // .....这里省略下订单的逻辑.....
23      state.products = [];
24    }
25  }
26 })
```

The background features a central horizontal band of vibrant blue. Within this band, numerous translucent spheres of varying sizes are scattered, some of which are brightly lit from within, creating a glowing effect. Horizontal white lines, resembling light rays or motion blur, pass through the center of the composition. The overall design is modern and geometric, with sharp diagonal lines separating the blue band from the lighter blue and grey areas above and below it.

Thank You !